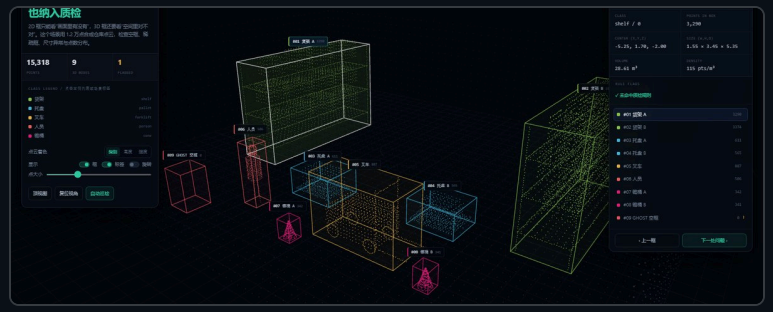
DATA ENGINEER / 3D VISION

把空间感，变成视觉数据的质量。

虚拟现实技术应用 · 专科应届。三个可交互作品：2D 标注质检、3D 点云检查、检测评估；外加一个数据审计 CLI。每个数字都能在本地复现。



01 / ANNOTATION QA



02 / 3D SPATIAL



03 / DETECTION EVAL

120 / 789	**24 / 24**	**15,318 / 9**	**0.757**
视觉数据 / 标注框	预置缺陷命中	点云 / 3D 框	mAP@0.5 / 96 图

我不跟本科拼算法，我跟他们拼“数据能不能落地”。

我关注实时三维视觉感知；
我做的是让 2D/3D 数据可采集、可标注、可质检。

01 / DATA

数据工程

120 张 / 5 类 / 789 框；采集、清洗、标注质检、报告、COCO 导出跑通一条链路。

02 / 3D

3D 空间

15,318 点 / 9 个 3D 框；框内点数、体积、密度、空框/稀疏/尺寸异常检查。

03 / EVAL

检测评估

96 图 / 641 GT；置信度、IoU、NMS 实时重算，FP/FN 定位到样本。

能力覆盖：采集/标注/质检、Linux/Python、AI 流程理解，分别由四个作品覆盖；3D 空间是差异化。C++/PyTorch 有基础，不包装成精通。

Python | Linux | COCO / YOLO | 标注质检 | 3D 点云 | mAP / IoU / NMS | Three.js | Git

能力 01

采集 / 标注 / 管理

120 张 / 5 类 / 789 框；YOLO/COCO 双格式；采集→清洗→标注→质检→导出。

能力 02

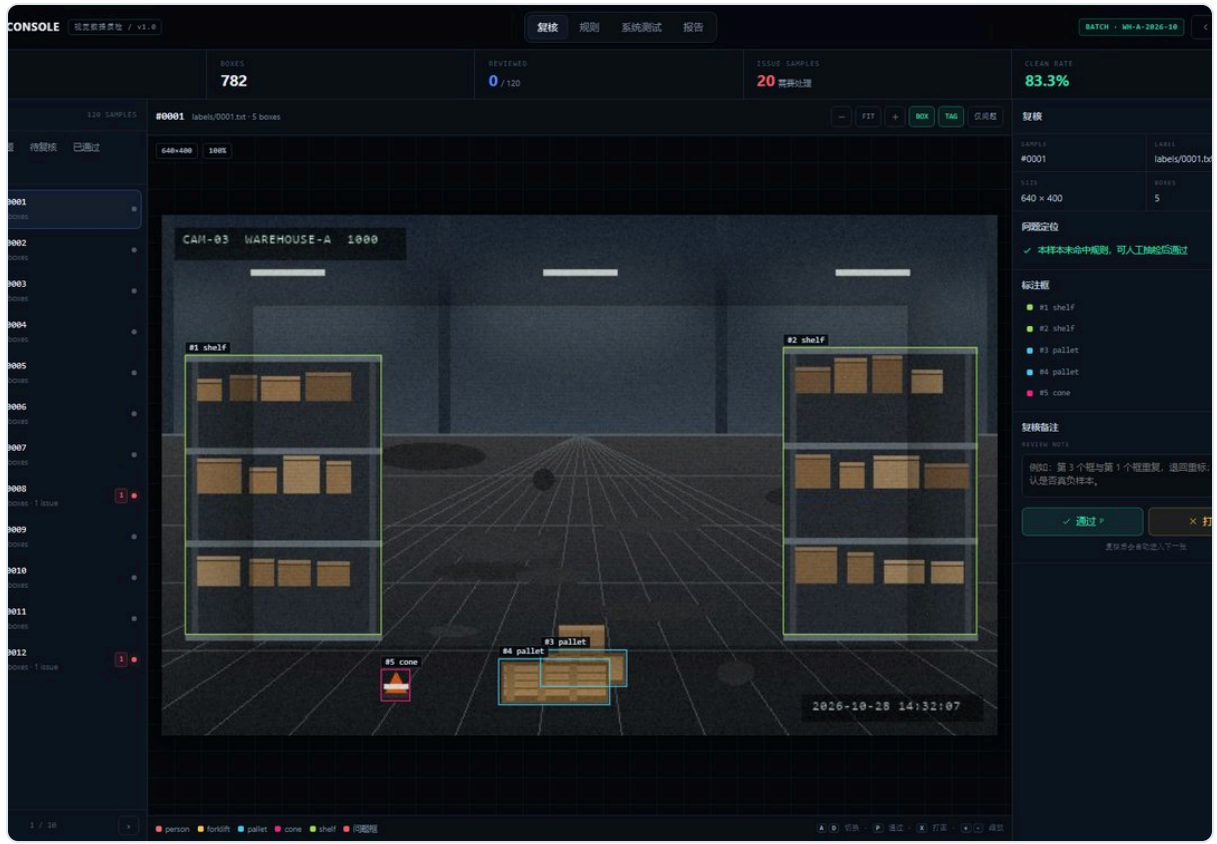
系统测试

8 条规则 + 24 个预置缺陷 24/24 命中；退回单能落到样本/框/规则。

能力 03

3D 空间

15,318 点 / 9 个 3D 框；空框/稀疏/尺寸异常检查，AABB 先验证。



ANNOTATION QA CONSOLE

视觉数据质检工作台

独立开发 · Python / YOLO / COCO / Canvas / Linux

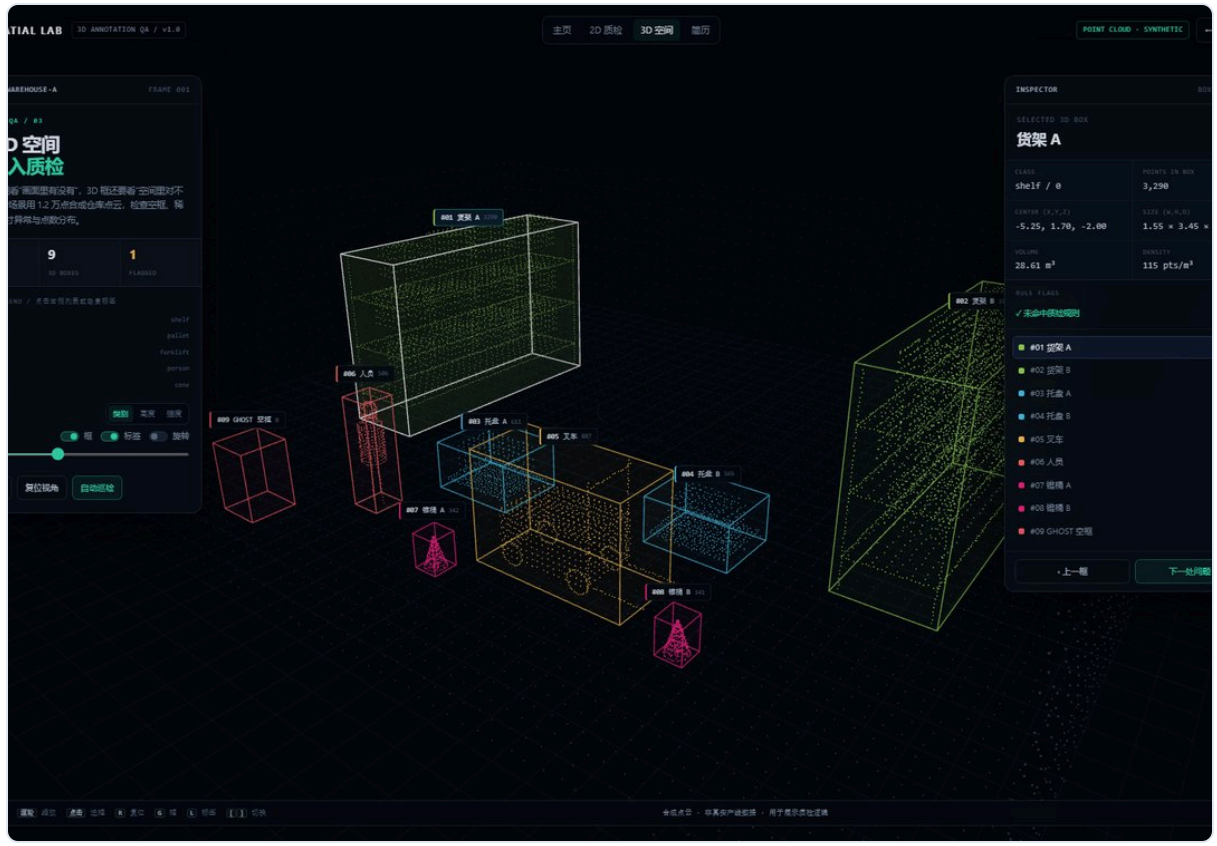
问题 标注数据里空标签、越界框、零面积、重复框、类别越界混在一起，只靠人眼看既慢又说不清"哪张图、哪个框"。

做法 自建 120 张 / 5 类 / 789 框数据；8 条规则；Canvas 逐样本逐框定位；通过/打回/备注/退回单；Python CLI 并发批处理。

结果 预置 24 个缺陷 `24/24` 命中，本批 0 误报；120 张批处理 `0.33s`；JSON/CSV/HTML 报告可直接交付。

120 / 789	24 / 24	0.33s
数据 / 框	缺陷命中	批处理

120 图 + YOLO → 解析 / 校验 → 8 条规则 → 逐框复核 → JSON / CSV / HTML



SPATIAL ANNOTATION INSPECTOR

3D 空间感知标注检查器

独立开发 · Three.js / 点云 / 3D BBox

问题

2D 框只能看"画面里有没有"，3D 框还要看"空间里对不对"。空框、稀疏框、尺寸异常很难靠肉眼发现。

做法

程序化生成 15,318 点仓库点云 + 9 个 3D 框；计算框内点数/体积/密度/长宽比；点击框/标签/列表联动；自动巡检。

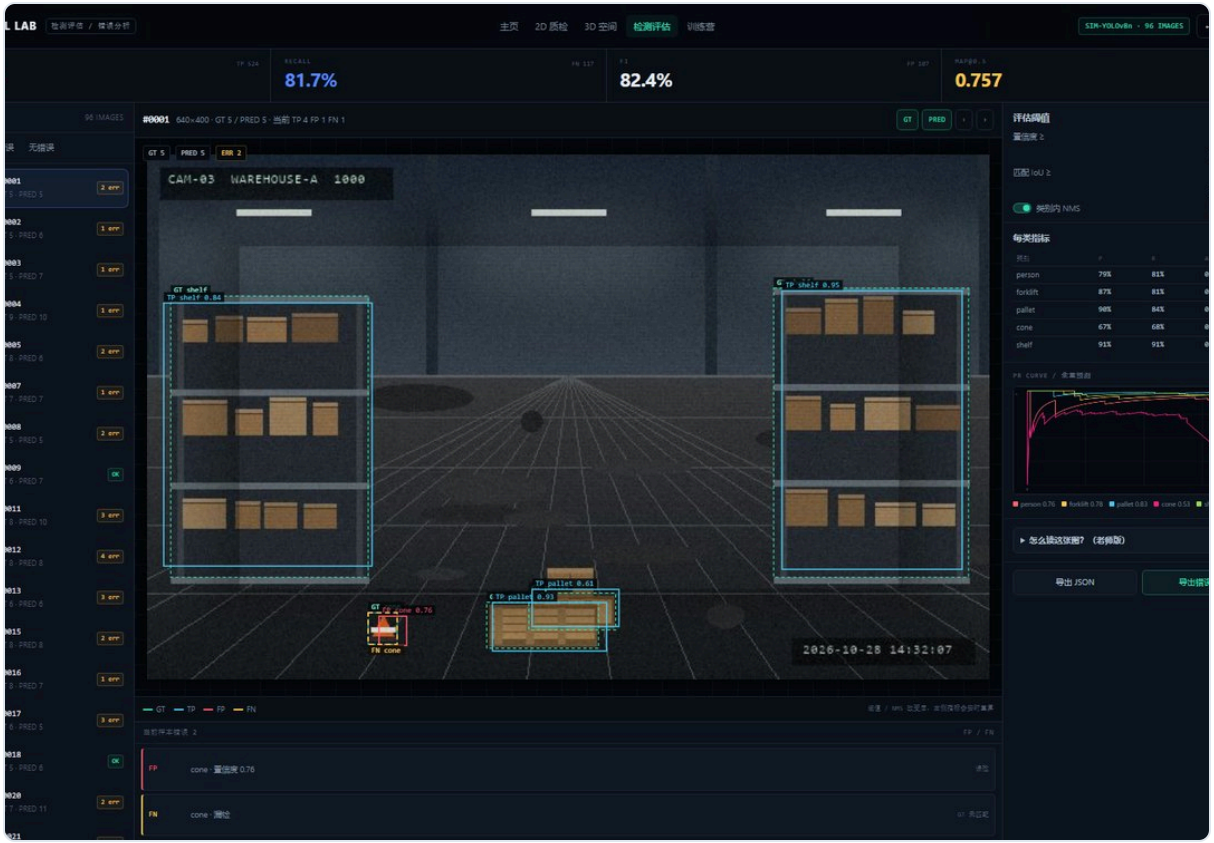
结果

检出 1 个 GHOST 空框；框内点数/密度一键可查；JSON 质检报告可直接复核。

15,318	9	1
点云	3D 框	空框

15,318 点云 → 9 个 3D 框 → 点数 / 密度 / 尺寸 → 空框 / 稀疏 / 异常 → JSON 报告

空间感 → 可检查指标 · AABB 先验证，真实用 OBB



DETECTION EVAL LAB

检测评估与错误分析

独立开发 · YOLO 输出 / mAP / NMS / Canvas

问题	只报一个 mAP，说不清误检/漏检在哪；调阈值、NMS 对指标的影响也讲不清。
做法	GT + 模拟 YOLO 输出；置信度/IoU/NMS 实时重算；TP/FP/FN 可视化；PR 曲线、每类 AP、错误清单。
结果	P `83.0%` / R `81.7%` / F1 `82.4%` / mAP@0.5 `0.757`；能解释阈值/NMS，FP/FN 定位到样本。

96 / 641
图 / GT

0.757
mAP@0.5

5 类
per-class AP

GT + Pred → IoU 匹配 → TP / FP / FN → P / R / AP → PR 曲线 + 错误清单

预测为模拟，评估逻辑真实 · 真实项目替换 `pred` 即可

不是“我会什么”，是“数据怎么从 A 到 B”

一条数据链路，四个可验证环节

采集 / 自建数据 → 解析 / 校验 → 标注质检 → 3D / 评估 → 报告 / 交付

DATA LAYER

数据层

Python / Pillow / JSON：生成图、解析 YOLO/COCO、跑审计 CLI。YOLO 归一化，COCO 像素，id 差 1。

WEB LAYER

页面层

HTML 骨架 / CSS 布局 / JS 行为；fetch 数据、Canvas 画框、localStorage 存复核状态。

3D LAYER

3D 层

Three.js：Scene/Camera/Renderer；BufferGeometry + Points 一次 draw call；Raycaster 选框。

面试一句话：“HTML 是骨架，CSS 是布局，JS 是行为；浏览器 fetch 标签 → 解析成框 → 规则/指标 → Canvas/WebGL 渲染 → 报告导出。”

Python | Pillow | JSON | HTML/CSS/JS | Canvas | Three.js | IoU / NMS / AP | Linux / Git

追问 01

为什么不用 React?

页面状态不复杂，原生 JS 足够；少一层框架，打开更快，也不依赖构建工具。

追问 02

15k 点怎么优化?

一个 BufferGeometry + 一个 Points，一次 draw call；不逐点建对象，标签用 HTML。

追问 03

数据 10 万张?

manifest + 分片 + 并发 + 断点续跑 + 版本化；先全量轻规则，再抽样重规则。

面试现场 30 秒

先看一个框，再看一张报告。

01 / QA Console

点一个红色问题样本，指问题列表："这是越界/重复框，能定位到第几个框。"

02 / Spatial Lab

点"自动巡检"，选中 GHOST 空框："空框为什么只标 REVIEW。"

03 / Eval Lab

置信度 0.25→0.60："P 升 R 降，阈值是业务取舍。"

01	02	03	04
作品集 / 演示 / 技术说明	现场可打开 Demo	深圳 · 可实习 · 随时到岗	GitHub / 作品集 / 深圳